



# Identity Verification Services

## Supplier Allocation Selector Design

### Table of Contents

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| 1. Overview .....                                                                                     | 3  |
| 2. Acceptance Criteria.....                                                                           | 3  |
| 3. Database Analysis .....                                                                            | 3  |
| 3.1 Create New Table: RP_PV_SUPPLIER_SELECTION .....                                                  | 3  |
| 4. Technical Specifications .....                                                                     | 4  |
| 4.1. Java Files Creation .....                                                                        | 4  |
| SupplierSelectionVo Properties: .....                                                                 | 5  |
| 4.2. Supplier Allocation Percentage Configuration .....                                               | 5  |
| 4.3. RpPvSupplierSelection Table Row Insertion for Every Configuration Change.....                    | 8  |
| 4.4. Main Supplier Determination Method .....                                                         | 9  |
| 4.5. Method Checking a Split Configuration .....                                                      | 9  |
| 4.6. Supplier Determination and Table Row Insertion for Each Transaction on Split Configuration ..... | 10 |

| <i><b>DOCUMENT IDENTIFICATION INFORMATION</b></i> |            |
|---------------------------------------------------|------------|
| <b>Version:</b>                                   | 1.0        |
| <b>Date Created:</b>                              | 05-04-2026 |
| <b>Created By:</b>                                | Emer Credo |
| <b>Date Published:</b>                            | 05-08-2026 |

| <b>CHANGE HISTORY</b> |             |                                    |               |
|-----------------------|-------------|------------------------------------|---------------|
| <b>Ver.</b>           | <b>Date</b> | <b>Change Description</b>          | <b>Author</b> |
| 1.0                   | 05/04/26    | Created initial draft of document. | Emer Credo    |

## 1. Overview

The current supplier allocation selector is causing production issue whenever the allocation of the suppliers is split especially for Change of Address. The issue is having delayed processing that a lot of times results in getting timed out. Every time a transaction is submitted, determination is being made which supplier would service the phone verification in the case when there are two or more suppliers are allocated. After a supplier is selected, the row of that supplier is updated to increment the Attempts count. While the row is updated, it is being locked by the current transaction and the subsequent transactions which are going to update the same row for update are waiting for the row be released. The same situation also happens to the other row for the other supplier once selected. The contention will create more blockers depending on the volume of transactions which can be very huge in production. Once the number of blockers and blockees grow, significant delay in transaction procession occurs.

## 2. Acceptance Criteria

1. Processing delay of Change of Address transaction should be reduced to an acceptable level (based on COA's normal and expected processing time) in production.
2. Database operation should only be limited to Select and Insert operations when determining the selected phone verification supplier. There should be no record update except when the supplier percentage allocation configuration is changed.  
RP\_KBA\_ATTEMPTS table is only updated on the TOTAL\_ATTEMPTS only during the IVSAdmin configuration change.

## 3. Database Analysis

### 3.1 Create New Table: RP\_PV\_SUPPLIER\_SELECTION

A new table will be created to hold the current percentages and attempts of the allocated suppliers.

| Column Name                  | Type             | Nullable | Data Default | Description                                                      |
|------------------------------|------------------|----------|--------------|------------------------------------------------------------------|
| <b>SUPPLIER_SELECTION_ID</b> | NUMBER<br>(18,0) | No       |              | Primary key. Sequence generated unique identifier.               |
| <b>PROOFING_LEVEL</b>        | NUMBER<br>(2,0)  | No       |              | LOA Proofing Level                                               |
| <b>SPONSOR_ID</b>            | NUMBER<br>(2,0)  | No       |              | Unique identifier for the sponsor and FK to ref_sponsor table.   |
| <b>SUPPLIER_ID_1</b>         | NUMBER<br>(2,0)  | No       |              | FK to REF_KBA_SUPPLIER for supplier #1.                          |
| <b>TOTAL_ATTEMPTS_1</b>      | NUMBER<br>(3,0)  | No       |              | Configured based on the allocation percentage of PV supplier #1. |

|                         |                 |     |        |                                                                                                        |
|-------------------------|-----------------|-----|--------|--------------------------------------------------------------------------------------------------------|
| <b>ATTEMPTS_1</b>       | NUMBER<br>(3,0) | No  |        | The current running attempt as incremented within the configured allocation percentage of supplier #1. |
| <b>SUPPLIER_ID_2</b>    | NUMBER<br>(2,0) | No  |        | FK to REF_KBA_SUPPLIER for supplier #2.                                                                |
| <b>TOTAL_ATTEMPTS_2</b> | NUMBER<br>(3,0) | No  |        | Configured based on the allocation percentage of PV supplier #2.                                       |
| <b>ATTEMPTS_2</b>       | NUMBER<br>(3,0) | No  |        | The current running attempt as incremented within the configured allocation percentage of supplier #2. |
| <b>SUPPLIER_ID_3</b>    | NUMBER<br>(2,0) | No  |        | FK to REF_KBA_SUPPLIER for supplier #3.                                                                |
| <b>TOTAL_ATTEMPTS_3</b> | NUMBER<br>(3,0) | No  |        | Configured based on the allocation percentage of PV supplier #3.                                       |
| <b>ATTEMPTS_3</b>       | NUMBER<br>(3,0) | No  |        | The current running attempt as incremented within the configured allocation percentage of supplier #3. |
| <b>CREATE_DATE</b>      | DATE            | No  |        | Date/time the record was inserted.                                                                     |
| <b>UPDATE_DATE</b>      | DATE            | Yes | (null) | Date/time the record was updated.                                                                      |

## 4. Technical Specifications

### 4.1. Java Files Creation

- A. A new RpPvSupplierSelection entity will be created for RP\_PV\_SUPPLIER\_SELECTION table with the following properties:

```

long supplierSelectionId
long proofingLevel
long sponsorId
long supplierId1
int totalAttempts1
int attempts1
long supplierId2
int totalAttempts2
int attempts2
long supplierId3
int totalAttempts3
int attempts3
Timestamp createDate
Timestamp updateDate

```

N.B. Setters and Getters will also be defined.

- B. Data access object and service layers will be created to support inserting of record into and retrieval of record from RP\_PV\_SUPPLIER\_SELECTION table.

DAO Files:

RpPvSupplierSelectionDao.java

RpPvSupplierSelectionDaoImpl.java

Service Files:

RpPvSupplierSelectionService.java

RpPvSupplierSelectionServiceImpl.java

- C. A value object, named SupplierSelectionVo, will be needed to hold ATTEMPTS and SUPPLIER\_ID that will be used as sort fields for each supplier. The supplier's Vo will be then added to a list which will be sorted being needed in populating RpPvSupplierSelection entity.

SupplierSelectionVo Properties:

```
long supplierId
int totalAttempts
int attempts
```

N.B. Setters and Getters will also be defined.

## 4.2. Supplier Allocation Percentage Configuration

- A. Either 100% or split configuration of supplier traffic allocation is still being done in the same IVS Admin UI. However, for split configuration, a record will be inserted for each configuration change to the new RP\_PV\_SUPPLIER\_SELECTION table in addition to the existing update operation to RP\_KBA\_ATTEMPTS table just on the TOTAL\_ATTEMPTS column for allocation percentage. For this new method, this will be the only time RP\_KBA\_ATTEMPTS table rows will be updated; ATTEMPTS column will not be updated anymore. For 100% configuration, no record will be inserted into the new RP\_PV\_SUPPLIER\_SELECTION table. This table is only for split configuraion.
- B. Sample Two-Supplier Configuration Change in SIT for COA.

B.1. Allocation Percentages: 60% Equifax, 40% LexisNexis

In this split configuration, two rows in RP\_KBA\_ATTEMPTS table will be updated as usual and one row in RP\_PV\_SUPPLIER\_SELECTION will be inserted.

B.2. Two rows will be updated RP\_KBA\_ATTEMPTS table with the following data:

| Column Name     | Row1 Value | Row2 Value |
|-----------------|------------|------------|
| KBA_SUPPLIER_ID | 5          | 4          |
| PROOFING_LEVEL  | 1          | 1          |
| SPONSOR_ID      | 32         | 32         |
| TOTAL_ATTEMPTS  | 60         | 40         |
| ATTEMPTS        | 0          | 0          |
| CREATE_DATE     | SYSDATE    | SYSDATE    |
| UPDATE_DATE     | SYSDATE    | SYSDATE    |

B.3. A new row will be inserted into RP\_PV\_SUPPLIER\_SELECTION with the following data:

| Column Name           | Row Value |
|-----------------------|-----------|
| SUPPLIER_SELECTION_ID | ?         |
| PROOFING_LEVEL        | 1         |
| SPONSOR_ID            | 32        |
| SUPPLIER_ID_1         | 5         |
| TOTAL_ATTEMPTS_1      | 60        |
| ATTEMPTS_1            | 0         |
| SUPPLIER_ID_2         | 4         |
| TOTAL_ATTEMPTS_2      | 40        |
| ATTEMPTS_2            | 0         |
| SUPPLIER_ID_3         | 0         |
| TOTAL_ATTEMPTS_3      | 0         |
| ATTEMPTS_3            | 0         |
| CREATE_DATE           | SYSDATE   |
| UPDATE_DATE           | SYSDATE   |

## C. Sample Three-Supplier Configuration Change in SIT for COA.

C.1. Allocation Percentages: 40% Equifax, 35% LexisNexis, 25% Experian

In this split configuration, three rows in RP\_KBA\_ATTEMPTS table will be updated as usual and one row in RP\_PV\_SUPPLIER\_SELECTION will be inserted.

C.2. Three rows will be updated RP\_KBA\_ATTEMPTS table with the following data:

| Column Name     | Row1 Value | Row2 Value | Row3 Value |
|-----------------|------------|------------|------------|
| KBA_SUPPLIER_ID | 5          | 4          | 6          |
| PROOFING_LEVEL  | 1          | 1          | 1          |
| SPONSOR_ID      | 32         | 32         | 32         |
| TOTAL_ATTEMPTS  | 40         | 35         | 25         |
| ATTEMPTS        | 0          | 0          | 0          |
| CREATE_DATE     | SYSDATE    | SYSDATE    | SYSDATE    |
| UPDATE_DATE     | SYSDATE    | SYSDATE    | SYSDATE    |

C.3. A new row will be inserted into RP\_PV\_SUPPLIER\_SELECTION with the following data:

| Column Name           | Row Value |
|-----------------------|-----------|
| SUPPLIER_SELECTION_ID | ?         |
| PROOFING_LEVEL        | 1         |
| SPONSOR_ID            | 32        |
| SUPPLIER_ID_1         | 5         |
| TOTAL_ATTEMPTS_1      | 40        |
| ATTEMPTS_1            | 0         |
| SUPPLIER_ID_2         | 4         |
| TOTAL_ATTEMPTS_2      | 35        |
| ATTEMPTS_2            | 0         |
| SUPPLIER_ID_3         | 6         |
| TOTAL_ATTEMPTS_3      | 25        |
| ATTEMPTS_3            | 0         |
| CREATE_DATE           | SYSDATE   |

### 4.3. RpPvSupplierSelection Table Row Insertion for Every Configuration Change

A. RpPvSupplierSelection entity will be instantiated, populated and inserted as follows:

A.1. A list of RpOtpAttemptConfig entities will be retrieved using the following query:

```
SELECT e FROM RpOtpAttemptConfig e WHERE e.id.proofingLevel = :proofingLevel
AND e.id.sponsorId = :sponsorId AND e.totalAttempts > 0 ORDER BY
e.refOtpSupplier.otpSupplierId
```

A.2. RpPvSupplierSelection entity properties will be populated as follows:

```
List< RpOtpAttemptConfig> list = resultList (from query in Subsection A.1.)
```

```
RpPvSupplierSelection entity = new RpPvSupplierSelection();
int index = 0;
```

```
for (RpOtpAttemptConfig c : list) {
    if (index = 0) {
        entity.setSupplierId1(c.getRefOtpSupplier().getOtpSupplierId());
        entity.setTotalAttempts1(c.getTotalAttempts());
        entity.setAttempts1(0);
    } else if (index = 1) {
        entity.setSupplierId2(c.getRefOtpSupplier().getOtpSupplierId());
        entity.setTotalAttempts2(c.getTotalAttempts());
        entity.setAttempts2(0);
    } else if (index = 2) {
        entity.setSupplierId3(c.getRefOtpSupplier().getOtpSupplierId());
        entity.setTotalAttempts3(c.getTotalAttempts());
        entity.setAttempts3(0);
    }
}
```

```
index++;
}
```

```
entity.setProofingLevel(1);
entity.setSponsorId(32);
entity.setCreateDate(new Timestamp(new Date().getTime()));
entity.setUpdateDate(new Timestamp(new Date().getTime()));
```

A.3. RpPvSupplierSelection entity will be persisted as follows:

RpPvSupplierSelectionService will be created (as described in Section 4.1.).



RpPvSupplierSelection entity will be populated (as detailed in Subsection A.2.)

RpPvSupplierSelection entity will be persisted using service as follows:

```
rpPvSupplierSelectionService.create(entity);
```

#### 4.4. Main Supplier Determination Method

A. A new main method will be created that will determine the supplier to service the current transaction.

B. Method Name and Parameters:

```
public RefOtpSupplier determineVerificationSupplier(PersonVo  
personVo, RefLoaLevel level, RefSponsor sponsor, List<Long>  
excludedSupplierIds)
```

C. Parameters:

PersonVo – this represents the transaction using its unique Person Id property

RefLoaLevel – LOA level of the transaction. Currently all remote proofing LOA is set to LOA 1.5 with an LOA Id of 1

RefSponsor – the sponsor of the transaction such as (Change of Address, CustReg, Operation Santa).

ExcludedSupplierIds – This the list of Supplier\_Id that will not be selected if available. IVS will select alternative supplier as fail-over if there is still available in the split configuration.

D. Conditions when a Supplier\_Id to the list:

1. If the result in Device Reputation Assessment is “Individual\_Not\_Found”. In this case, LexisNexis Id is added to the excluded list.
2. If Experian is selected to service Phone Verification and the customer is evaluated as “Customer-Not-On-File”, the main method is called again to get alternative supplier and Experian is added to the excluded list.
3. If the selected Phone Verification supplier encounters processing error such as when the API call does not return any response (i.e. null response) or the response contains error code or message and the transaction decision cannot be obtained due to network error or timed out error, the Supplier\_id is added to the excluded list.

#### 4.5. Method Checking a Split Configuration

- A. This method is an existing method to check Rp\_Kba\_Attempts table if a configuration of the sponsor is split, that is no supplier is allocated a 100% traffic. Otherwise, the supplier with 100% allocation is selected.

- B. Method Name and Parameters:

```
private RpOtpAttemptConfig getSoleOtpAttemptConfig(RefLoaLevel
level, long sponsorId) {
    RpOtpAttemptConfig soleOtpAttemptConfig = null;
    List<RpOtpAttemptConfig> list =
        otpAttemptConfigService.getByProofingLevel(
            level.getLoaCode(), sponsorId);

    for (RpOtpAttemptConfig otpAttemptConfig : list) {
        if (otpAttemptConfig.getTotalAttempts() == 100) {
            soleOtpAttemptConfig = otpAttemptConfig;
            break;
        }
    }

    return soleOtpAttemptConfig;
}
```

- C. Parameters:

RefLoaLevel – LOA level of the transaction. Currently all remote proofing LOA is set to LOA 1.5 with an LOA Id of 1

RefSponsor Id – the sponsor Id from the RefSponsor entity passed in the main method.

- E. If the method in Subsection B will determine a 100 % allocation configuration, a RpOtpAttemptConfig entity will be returned as the only available entity. The supplier associated with this entity will be selected.

#### 4.6. Supplier Determination and Table Row Insertion for Each Transaction on Split Configuration

- A. The latest record of RP\_PV\_SUPPLIER\_SELECTION will be retrieved using the following query.

```
SELECT e FROM RpPvSupplierSelection e WHERE ROWNUM = 1 ORDER BY
e.createDate DESC
```

- B. If the SupplierId is not in the ExcludedSupplierIds (as defined in Section 4.3.), SupplierSelectionVo is populated and added to the list.

RpPvSupplierSelection selectionEntity (from query in Subsection A)

```
List<SupplierSelectionVo> selectionVoList = new ArrayList<>();

if (!excludedSupplierIds.isEmpty()) {
    if (!excludedSupplierIds.contains(selectionEntity.getSupplierId1())) {
        SupplierSelectionVo selectionVo1 = new SupplierSelectionVo();
        selectionVo1.setSupplierId(selectionEntity.getSupplierId1());
        selectionVo1.setTotalAttempts(selectionEntity.getTotalAttempts1());
        selectionVo1.setAttempts(selectionEntity.getAttempts1());
        selectionVoList.add(selectionVo1);
    }

    if (!excludedSupplierIds.contains(selectionEntity.getSupplierId2())) {
        SupplierSelectionVo selectionVo2 = new SupplierSelectionVo();
        selectionVo2.setSupplierId(selectionEntity.getSupplierId2());
        selectionVo2.setTotalAttempts(selectionEntity.getTotalAttempts2());
        selectionVo2.setAttempts(selectionEntity.getAttempts2());
        selectionVoList.add(selectionVo2);
    }

    if (!excludedSupplierIds.contains(selectionEntity.getSupplierId3())) {
        SupplierSelectionVo selectionVo3 = new SupplierSelectionVo();
        selectionVo3.setSupplierId(selectionEntity.getSupplierId3());
        selectionVo3.setTotalAttempts(selectionEntity.getTotalAttempts3());
        selectionVo3.setAttempts(selectionEntity.getAttempts3());
        selectionVoList.add(selectionVo3);
    }
}
```

- C. SupplierSelectionVo list is sorted by attempts, then by supplierId. The list is sorted in such a way the supplier with the lowest Attempts value is at the top. In the case of same Attempts values, the supplier with the lowest SupplierId is at the top. The default sort order for both Attempts and SupplierId is ascending.

List<SupplierSelectionVo> unsortedVoList (collected from Subsection B)

```
List<SupplierSelectionVo> sortedVoList = unsortedVoList.stream()
    .sorted(Comparator.comparingInt(SupplierSelectionVo::getAttempts)
        .thenComparing(SupplierSelectionVo::getSupplierId))
    .collect(Collectors.toList());
```

- D. The selected SupplierSelectionVo is determined by comparing the Attempts value against the Total Attempts value. The first SupplierSelectionVo in the list whose Attempts value is lower than the TotalAttempts value is selected. Once the Attempts value of SupplierSelectionVo item has reached the TotalAttempts value, it is not selected anymore. When all the SupplierSelectionVo items have reached the TotalAttempts values, the top in the list, which is the one with the lowest SupplierId value, is selected.

```

List<SupplierSelectionVo> sortedVoList (collected from Subsection C)
SupplierSelectionVo selectedVo = null;

for (SupplierSelectionVo vo : sortedVoList) {
    if (vo.getAttempts() < vo.getTotalAttempts()) {
        selectedVo = vo;
        break;
    }
}

if (selectedVo == null) {
    SelectedVo = sortedVoList.get(0);
}

```

- E. The Attempts value of the selected SupplierSelectionVo, whose Attempts value is lower than the TotalAttempts value, is incremented and a new RpPvSupplierSelection record is created, populated and inserted.

```

List< SupplierSelectionVo> sortedVoList (collected from Subsection C)

RpPvSupplierSelection prevEntity (from Subsection A – latest
                                RpPvSupplierSelection)
SupplierSelectionVo selectedVo (from Subsection D)
long selectedSupplierId = selectedVo.getSupplierId();

RpPvSupplierSelection newEntity = new RpPvSupplierSelection();

if (prevEntity.getSupplierId1() == selectedSupplierId) {
    newEntity.setAttempts1(prevEntity.getAttempts1() + 1);
} else if (prevEntity.getSupplierId2() == selectedSupplierId) {
    newEntity.setAttempts2(prevEntity.getAttempts2() + 1);
} else if (prevEntity.getSupplierId3() == selectedSupplierId) {
    newEntity.setAttempts3(prevEntity.getAttempts3() + 1);
}

newEntity.setSupplierId1(prevEntity.getSupplierId1());
newEntity.setTotalAttempts1(prevEntity.getTotalAttempts1());
newEntity.setSupplierId2(prevEntity.getSupplierId2());
newEntity.setTotalAttempts2(prevEntity.getTotalAttempts2());
newEntity.setSupplierId3(prevEntity.getSupplierId3());
newEntity.setTotalAttempts3(prevEntity.getTotalAttempts3());

newEntity.setProofingLevel(prevEntity.getProofingLevel());
newEntity.setSponsorId(prevEntity.getSponsorId());
newEntity.setCreateDate(new Timestamp(new Date().getTime()));
newEntity.setUpdateDate(new Timestamp(new Date().getTime()));

rpPvSupplierSelectionService.create(newEntity);

```

- F. When all the SupplierSelectionVo items have reached the TotalAttempts values, the top in the list is selected and all the Attempts values are reset back to 1 for the selected while the rest of the SupplierSelectionVo items are reset back to 0.

```
SupplierSelectionVo selectedVo (from Subsection D – the top of the list when all
                                items have reached the TotalAttempts)
long selectedSupplierId = selectedVo.getSupplierId();

RpPvSupplierSelection newEntity = new RpPvSupplierSelection();
newEntity.setAttempts1(0);
newEntity.setAttempts2(0);
newEntity.setAttempts3(0);

if (prevEntity.getSupplierId1() == selectedSupplierId) {
    newEntity.setAttempts1(1);
} else if (prevEntity.getSupplierId2() == selectedSupplierId) {
    newEntity.setAttempts2(1);
} else if (prevEntity.getSupplierId3() == selectedSupplierId) {
    newEntity.setAttempts3(1);
}

newEntity.setSupplierId1(prevEntity.getSupplierId1());
newEntity.setTotalAttempts1(prevEntity.getTotalAttempts1());
newEntity.setSupplierId2(prevEntity.getSupplierId2());
newEntity.setTotalAttempts2(prevEntity.getTotalAttempts2());
newEntity.setSupplierId3(prevEntity.getSupplierId3());
newEntity.setTotalAttempts3(prevEntity.getTotalAttempts3());

newEntity.setProofingLevel(prevEntity.getProofingLevel());
newEntity.setSponsorId(prevEntity.getSponsorId());
newEntity.setCreateDate(new Timestamp(new Date().getTime()));
newEntity.setUpdateDate(new Timestamp(new Date().getTime()));

rpPvSupplierSelectionService.create(newEntity);
```

- G. For the next transaction, the selected supplier is determined by repeating the steps in this Section 4.6.